

LeadForge AI — Project Documentation

Overview

LeadForge AI is a B2B lead extraction and email outreach SaaS platform. It automatically discovers leads by crawling the web, verifies their email addresses, enriches company data via AI, scores leads against your Ideal Customer Profile (ICP), and sends multi-step email campaigns — all from a single dashboard.

Tech Stack

Layer	Technology
Frontend	Next.js 14 (App Router), React 18, TypeScript, Tailwind CSS, shadcn/ui
Backend	Node.js 20, Express, TypeScript, Prisma ORM
Database	PostgreSQL 16
Cache / Queue	Redis 7 + BullMQ
Scraping	Playwright, Cheerio
Auth	Clerk (JWT + API keys)
AI	OpenAI GPT-4o-mini / Groq Llama 3.3 (LLM scoring, NER)
Billing	Stripe
Dev Infra	Docker Compose
Prod Infra	Vercel (frontend) + Railway/AWS ECS (backend)

Repository Layout

```
LeadForge-AI/
├─ backend/

|   └─ src/

|       └─ api/           Express REST API + SSE

|       └─ filters/       Recursive filter engine (AND/OR/NOT)

|       └─ intelligence/  Lead enrichment + AI scoring pipeline

|       └─ scraping/      Web scraping + lead extraction

|       └─ services/      Email sending, CRM, OAuth, LLM router

|       └─ verification/  6-layer email verifier

|       └─ workers/       BullMQ background workers

|   └─ prisma/            Database schema + migrations

├─ frontend/

|   └─ src/

|       └─ app/           Next.js pages (dashboard, auth, public)

|       └─ components/    Reusable UI components

|       └─ lib/           API client, utilities

├─ docs/                 Architecture, API spec, schema, wireframes

├─ infrastructure/

|   └─ docker/            Docker Compose for local dev
```

Core Features

1. Lead Extraction

Users create an `ExtractionJob` by selecting source types and a filter tree. The job runs asynchronously through BullMQ:

- `POST /jobs` → validates filters, creates DB record, enqueues `extraction` job
- **Extraction worker** discovers URLs via search engines, scrapes pages with Playwright/Cheerio, extracts lead data, and deduplicates by `(teamId, emailNormalized)`
- Progress is published to Redis pub/sub and streamed to the UI via SSE (`/sse`)
- Each extracted lead is automatically enqueued for **verification** and **enrichment**

Source types: `WEB_SEARCH`, `DIRECTORY`, `COMPANY_PAGE`, `BLOG`, `FORUM`, `SOCIAL_LINKEDIN`, `CONTACT_PAGE`, `LISTING`, `CUSTOM_URL_LIST`, and more. **Scheduling:** Jobs support cron expressions (e.g. `"0 9 * * 1"` for Monday 9am recurring runs).

2. Email Verification (6 Layers)

Every discovered email goes through a pipeline:

Stage	Method	Cached
1	RFC syntax check	—
2	Disposable domain detection	static set
3	MX record lookup	PostgreSQL DomainCache (7-day TTL)
4	SMTP RCPT TO probe	—
5	Catch-all domain probe	inferred
6	Role-account flag (info@, admin@, etc.)	—

Result: score 0–100, status `VALID` | `INVALID` | `RISKY` | `CATCH_ALL` | `UNKNOWN`, stored in `EmailVerification` with 30-day TTL.

3. Intelligence Pipeline

Each lead runs through the orchestrator which executes three steps in parallel:

- **Company enrichment** — fetches firmographics (industry, employee count, tech stack, headquarters)
- **Intent signal detection** — scans for hiring signals, funding announcements, executive moves
- **Identity resolution** — fuzzy-matches against existing leads in the workspace to detect duplicates

Results are written into a **Postgres-backed knowledge graph** (`Company` , `Industry` , `Technology` , `FundingEvent` , `ExecutiveMove` tables). Leads link to canonical `Company` nodes via `Lead.companyId` .

Scoring — Ensemble Model:

```
Heuristic scorer (fast, deterministic)
+ LLM scorer (GPT-4o-mini / Groq, ICP-aware narrative judgment)

→ Adaptive weighted average (rich signal → trust heuristic more)

→ Score 0–100, Tier A / B / C / D
```

Key heuristic weights: verified email (+12), C-level seniority (+15), recent funding in 90d (+12), recent exec change (+12), public company (+5), free email provider (−5), invalid email (−30).

4. Filter Engine

A recursive boolean filter tree:

```
Condition | { AND: FilterNode[] } | { OR: FilterNode[] } | { NOT: FilterNode }
```

- `compileFilterToWhere()` → Prisma `LeadWhereInput` for database queries
- `matchInMemory()` → boolean predicate for stream-filtering during scraping
- 25+ supported fields: niche, location, company size, role, seniority, technology stack, social presence, scores

5. Campaign Engine

Multi-step email drip campaigns with:

- **Sending accounts:** SMTP, SendGrid, AWS SES, Gmail OAuth, Microsoft OAuth (Outlook)

- **Multi-step sequences:** Ordered steps with per-step day delays and `{{variable}}` templates
- **Scheduling windows:** send hours, weekdays-only toggle, daily send limit per account
- **Reply detection:** IMAP polling worker matches inbound `Message-ID` headers to sent emails
- **Engagement tracking:** Opens (pixel), clicks (link rewriting), bounces, replies, unsubscribes
- **Live monitor:** SSE-powered real-time campaign stats

6. CRM Integrations

Leads can be pushed to **HubSpot**, **Salesforce**, or **Pipedrive**. Each push is logged with the remote contact ID.

7. Exports

Leads export as `CSV` , `XLSX` , or `JSON` . Large exports go through the BullMQ export worker, producing files uploaded to S3 with 7-day signed URLs. Small CSV exports stream directly from the API.

8. Webhooks

Teams register webhook URLs for events. Deliveries are queued via BullMQ and signed with HMAC-SHA256 (`X-LeadForge-Signature` header).

Background Workers

All run as a separate Node.js process (`npm run worker`) backed by BullMQ + Redis:

Worker	Responsibility
<code>extraction.worker</code>	Crawls sources, extracts + deduplicates leads
<code>verification.worker</code>	Runs 6-layer email verification per lead
<code>enrichment.worker</code>	Triggers the full intelligence pipeline per lead
<code>campaign.worker</code>	Sends emails per campaign schedule, respects daily limits
<code>imap-poll.worker</code>	Polls IMAP inboxes for replies, updates send records
<code>export.worker</code>	Generates and uploads CSV/XLSX/JSON exports
<code>webhook.worker</code>	Delivers signed webhook payloads with retry logic

API Routes

Prefix	Description
POST /auth/*	Clerk webhook sync
GET/POST /leads	List, search, create, export leads
GET/POST /jobs	Extraction job CRUD + SSE progress
GET/POST /campaigns	Campaign CRUD + launch/pause
GET/POST /lists	Lead list management
GET/POST /sending-accounts	Email sending account setup + OAuth flows
GET/POST /integrations	CRM connection CRUD
GET /analytics	Dashboard stats
POST /verification	Manual email verification trigger
GET/POST /exports	Async export management
GET/POST /webhooks	Webhook CRUD
GET/POST /api-keys	API key generation/revocation
POST /assistant	AI assistant (natural-language lead queries)
GET /sse	Server-sent events for job progress
GET /health, /ready	Liveness + readiness probes

Authentication

- **Dashboard:** Clerk JWT (`Authorization: Bearer`)
- **API access:** `lf_<32-byte-base64url>` keys — SHA-256 hashed at rest; plaintext returned only at creation. Keys have scopes and optional expiry.
- **Multi-tenancy:** Every query is scoped to `teamId` . The `X-Team-Id` header switches active team for users in multiple workspaces.
- **RBAC:** `OWNER` | `ADMIN` | `MEMBER` | `VIEWER` roles enforced via middleware.

Data Model Summary

```
User —< TeamMembership >— Team
Team —< ExtractionJob —< Lead —< EmailVerification
                                —< LeadEnrichment
                                —< CrmPush

Lead >— Company —< Technology
                        —< FundingEvent
                        —< ExecutiveMove

Team —< Campaign —< CampaignStep
                        —< CampaignRecipient —< EmailSend —< EmailEvent

Team —< LeadList —< LeadListMembership

Team —< SendingAccount

Team —< CrmConnection

Team —< Webhook

Team —< LeadScoreFeedback    (ML training labels)

CrawlEntry / CrawlHost      (distributed crawler bookkeeping)
```

Frontend Pages

Route	Page
<code>/dashboard</code>	Overview stats + recent jobs
<code>/extraction</code>	Create/configure extraction jobs
<code>/jobs</code>	Job list + <code>/jobs/[id]</code> live progress
<code>/leads</code>	Lead table with filters, bulk actions, CSV export
<code>/search</code>	Knowledge graph search with score-feedback bar
<code>/lists</code>	Lead list management
<code>/companies</code>	Company knowledge graph browser
<code>/campaigns</code>	Campaign list + new wizard + live monitor
<code>/verification</code>	Manual email verification
<code>/analytics</code>	Charts (leads over time, verification breakdown, etc.)
<code>/assistant</code>	AI chat for natural-language lead queries
<code>/settings/integrations</code>	CRM + webhook connections
<code>/settings/sending-accounts</code>	Email sending account setup
<code>/settings/icp</code>	ICP profile editor (used by AI scorer)
<code>/settings/api-keys</code>	API key management
<code>/settings/billing</code>	Stripe subscription management
<code>/admin</code>	Internal admin panel

Local Development

```
# 1. Start PostgreSQL + Redis
docker compose -f infrastructure/docker/docker-compose.yml up -d
```

2. Backend API (port 4000)

```
cd backend
```

```
cp .env.example .env # fill in secrets
```

```
npm install
```

```
npm run db:migrate
```

```
npm run db:seed # optional demo data
```

```
npm run dev
```

3. Background workers (separate terminal)

```
cd backend && npm run worker
```

4. Frontend (port 3000)

```
cd frontend
```

```
cp .env.example .env.local
```

```
npm install
```

```
npm run dev
```

API: <http://localhost:4000> · Dashboard: <http://localhost:3000>

Key Environment Variables

Variable	Used by
<code>DATABASE_URL</code>	Prisma — PostgreSQL connection string
<code>REDIS_URL</code>	BullMQ + rate limiter
<code>CLERK_SECRET_KEY</code>	JWT verification middleware
<code>OPENAI_API_KEY</code>	LLM scorer + NER extractor
<code>GROQ_API_KEY</code>	Fallback LLM (Llama 3.3)
<code>STRIPE_SECRET_KEY</code>	Billing
<code>STRIPE_WEBHOOK_SECRET</code>	Webhook signature verification
<code>AWS_S3_BUCKET</code>	Export file storage
<code>GOOGLE_CLIENT_*</code>	Gmail OAuth
<code>MICROSOFT_CLIENT_*</code>	Outlook OAuth

Production Deployment

- **Frontend:** Vercel (auto-deploys from `main`)
- **API + workers:** Railway (separate `api` and `worker` services)
- **Database:** Neon or RDS PostgreSQL
- **Cache/queue:** Upstash Redis
- **Exports:** AWS S3 with 7-day signed URLs

Compliance & Security

- `robots.txt` respected by the scraper (cached per origin)
- GDPR: right-to-delete on User + Lead, verification source URL stored for traceability, full audit log
- Webhook payloads signed with HMAC-SHA256
- Rate limiting: 100 req/min per user/IP via Redis
- API keys hashed at rest with SHA-256; plaintext shown only once at creation
- Lead extraction is unlimited for all users (credits system retained for future metering only)